

ONE-SITE & YARDI REPORTING PORTAL

Technical Scope, Mac Execution Rationale, and In-House Software Portfolio Value

Prepared for	Marc Menowitz, CEO — ApartmentCorp
Prepared by	Brandon Rose, Special Projects
Date	September 1, 2026

Executive conclusion: The portal remains accessible from any authorized computer or phone. The Mac is the correct provider-execution workstation today because the current OneSite runner was built and validated for macOS and a visible Microsoft Edge session. The Windows work computer remains a viable future path, but moving to it first requires a separate, reviewed, and policy-compatible Windows bridge. The total in-house software portfolio represented here has a conservative external-development value of approximately **\$178,125–\$263,400** at \$75/hour, with a midpoint of roughly **\$221,000**.

Prepared for internal leadership review

Prepared for: Marc Menowitz, CEO — ApartmentCorp

Prepared by: Brandon Rose, Special Projects

Date: September 1, 2026

Purpose: This document answers two questions directly. First, what the OneSite & Yardi Reporting Portal actually entails — why it is a multi-week engineering program and not a one-week task. Second, what the entire portfolio of internal applications I have built for the company would have cost if we had paid an outside developer to build them, estimated conservatively at **\$75/hour**.

Part 1 — What the OneSite & Yardi Reporting Portal Actually Entails

1.1 What the product is

The OneSite & Yardi Reporting Portal (deployed as the **OneSite Reporting Hub**) is a production internal management platform — explicitly not a prototype — that lets authorized management users request, generate, file, review, and distribute operational reports from two separate property-management systems, **RealPage OneSite** and **Yardi**, from any computer or phone, including iPhone and Google Fold devices.

The critical architectural fact — and the reason this project is measured in weeks rather than days — is that **neither RealPage OneSite nor Yardi offers a public API for report automation** [1] [2]. Both vendors expose data only through interactive, authenticated web sessions protected by CAPTCHA and SMS two-factor verification. Everything the portal does must therefore be built around a secure, auditable bridge between a normal web application and a human-signed-in browser session, without ever touching, storing, or transmitting provider credentials. That constraint drives the entire design.

1.2 The eight major subsystems

The portal is not one application; it is eight coordinated subsystems, each of which is a meaningful software project on its own.

#	Subsystem	What it does
1	Responsive web portal	The management-facing application: individual user accounts, source-specific access control, audit visibility, request creation, status tracking, and a full report library. Works on desktop and mobile.
2	Live report catalog synchronization	The OneSite Classic Reports catalog — 310 reports across 10 paginated pages — is scraped from a live Edge session and kept in sync so the portal's report selector always reflects the real provider catalog. A separate, fully isolated Yardi catalog flow is in staged development.

#	Subsystem	What it does
3	Property directory synchronization	The authoritative property list is pulled from OneSite's Classic-mode property dropdown and synchronized into the portal's property directory, covering the full 42-property portfolio.
4	Request workflow engine	Users select a report, set report-specific parameters (e.g., "Delinquent Rent Only: On"), and run it for all properties or one selected property . All-property runs must drive OneSite's explicit *Select All* behavior — not a visible-page subset — which required building a dedicated preflight and inspection layer.
5	Self-hosted runner infrastructure	Provider-side work is executed by a private GitHub Actions self-hosted runner (BrandonRose2/realpage-portal-runner) registered to a designated management computer. The runner claims queued requests, verifies a live authenticated Edge session, drives report generation, and downloads completed outputs.
6	Document filing and storage pipeline	Every output is filed under separate source roots — OneSite Reporting/{Property}/... and Yardi Reporting/{Property}/... — preserving original Excel/PDF files, request metadata, status, and generated HTML summaries, with duplicate protection and idempotent filing.
7	Report library and viewers	Filed reports are filterable and sortable by source, date, property, and report type. PDFs open in an in-portal viewer; Excel workbooks get a sheet-aware preview plus original-file download; every workbook also gets a polished, mobile-readable responsive HTML companion generated automatically.
8	Manager workflow and communications	The portal generates property-specific, editable Manager's Checklists (HTML/Markdown) with per-line verification, auto-save, progress bars, "Needs attention" deep links, and a "Submit for Review" flow; matches property managers and regional managers against the company contacts directory with phone, extension, and email; and drafts (never auto-sends) review emails.

1.3 The security model — and why it is non-negotiable

Because OneSite and Yardi sessions are protected by CAPTCHA and SMS two-factor authentication, the portal is built around a hard security boundary that a commercial vendor would treat as a compliance requirement:

The portal, the runner, the repository, the logs, and the chat never collect, store, relay, or expose OneSite or Yardi usernames, passwords, browser cookies, session tokens, or MFA codes. Provider login and CAPTCHA/2FA completion remain human actions inside a provider-approved, visible Microsoft Edge session on a designated management computer.

Concretely, this required: a credential-free runner configuration (a protected local file containing only the portal URL, a runner token, and a live-session flag — kept out of Git); a narrowly scoped, temporary Microsoft Edge companion extension limited to the My Reports download context for a single approved batch; complete **provider separation** (OneSite and Yardi have distinct sessions, runners, catalogs, request handling, storage roots, and tokens); and fail-closed reconciliation logic that refuses to act unless request name, format, property counts, and filing pairs all match.

1.4 The current engineering problem: Request #60001 and the Windows transition

The exact recovery point the project sits at today illustrates why this is real engineering rather than button-clicking:

- **Request #60001** — the "All Units (Excel)" portfolio run dated August 28, 2026 — used OneSite's Select All across **35 selected property options**, producing results across multiple paginated My Reports pages. Ten original workbooks and ten HTML companions were filed before the run froze.
- The post-run downloader repeatedly scanned only the **first ten visible rows** of the paginated My Reports grid and failed to wait for page transitions, so the remaining completed outputs were not reliably captured.
- A later Edge companion was restricted to a **21-property filing plan**, which conflicts with the recorded 35-option selection. That discrepancy **cannot safely be resolved by rerunning the report** — rerunning would create duplicate provider-side results and destroy the audit trail. It must be reconciled forensically, by building a single truth table across the provider's My Reports entries, the portal's document records, and the runner's staged files, then filing only genuinely missing, already-completed original/HTML pairs.
- Meanwhile, the only registered execution machine (a macOS self-hosted runner using macOS-specific AppleScript tab control) went offline, and my day-to-day workstation is now a **company-managed Windows PC**. Because the existing automation layer is macOS-specific, a **separate, security-reviewed Windows Edge bridge** had to be designed: a Microsoft Edge extension plus a local native-messaging host, pinned to one exact extension ID, with a versioned, nonce-protected, schema-validated message protocol, an eight-stage validation sequence (environment preflight → policy check → loopback test → tab readiness → source readiness → read-only inventory → non-duplicative filing → Yardi discovery), and a self-service, no-admin installer package whose compiled executable imports only `KERNEL32.dll` and `msvcrt.dll`.

All of this work — the recovery audit, the queue cleanup, the Windows preflight branch, the loopback bridge, the installer, and 15+ automated security tests — has been completed and validated **without a single provider action**: no report rerun, no email, no settings change, no credential exposure. That is what "doing it safely" looks like when the systems involved hold the company's live financial and resident data.

1.5 Why the Mac is the better execution environment for this reporting workflow — at this stage

The point is **not** that a Mac is universally better than a Windows computer for all ApartmentCorp work. The web portal itself is intentionally device-independent: authorized managers can use it from a Windows PC, a Mac, an iPhone, or a Google Fold. The distinction is limited to the highly controlled **provider-side execution environment** for RealPage OneSite and, later, Yardi.

For the current reporting workflow, the Mac is the safer and more efficient choice because it is the environment for which the operational runner was actually built, registered, and tested. The existing self-hosted runner is a macOS/ARM64 runner with the required self-hosted, macOS, and onesite-runner labels; its OneSite integration uses macOS-specific AppleScript controls to communicate only with an **already open, visible Microsoft Edge tab** in Brandon's signed-in desktop session. It does not launch a separate browser profile, store credentials, or attempt to bypass OneSite's CAPTCHA or two-factor authentication.

Decision factor	Existing Mac execution path	Windows work-computer path today
Runner compatibility	The existing runner and GitHub Actions workflow were built and labeled for macOS.	The current production workflow will not route to Windows; it requires a separately labeled Windows workflow.
Visible Edge control	The current implementation uses a tested macOS-native AppleScript control layer for the user's visible Edge tab.	Windows requires a new Edge extension + native-messaging bridge because macOS AppleScript cannot run on Windows.
Security and credential boundary	Uses the already authenticated visible Edge session while keeping all credentials, cookies, tokens, and MFA information outside the portal and repository.	Can achieve the same boundary, but only after the new bridge, extension identity pinning, policy checks, installer, and readiness tests are completed and reviewed.
Operational readiness	The architecture exists now; the remaining Mac issue is a repair of the existing GitHub runner host (Runner.Listener is missing), not a redesign of provider interaction.	The Windows approach is in staged readiness design. Its loopback package is deliberately not yet a production OneSite integration and cannot reconcile Request #60001.
Company-device policy	The Mac path uses the already-defined management-machine boundary.	The Windows machine is company-managed, so its native host/extension must also satisfy local enterprise policy or obtain a narrowly scoped administrator approval.
Risk and time to resume	Repairing the runner preserves the previously tested code path and requires the fewest moving parts.	Moving immediately to Windows adds an implementation, packaging, security-review, installation, and validation project before any report work can resume.

Bottom line: For OneSite reporting today, continuing with the Mac is not a preference for one brand of computer. It is the lowest-risk route because it preserves a tested, credential-free, provider-safe execution path. Using Windows before the dedicated Windows bridge is fully built and validated would mean replacing the automation layer in the middle of a live-report reconciliation — exactly the type of avoidable change that creates reporting errors, duplicate files, and security risk.

The correct operating model is therefore straightforward: managers can create, review, and access reports from **any device** through the portal, while the Mac temporarily remains the designated **provider-execution workstation** until the existing runner is repaired and, if desired, the separate Windows bridge passes its staged security and readiness gates. This is a deliberate continuity and risk-management decision, not a limitation on management access.

1.6 Why "less than a week" was never a realistic expectation

To put the scope in market terms, the table below breaks the portal into the work packages an outside development shop would quote, at the requested **\$75/hour** blended rate. These are conservative midpoints for a senior full-stack developer; agencies typically quote \$125–\$200/hr for equivalent integration work [3] [4].

Work package	Est. hours	Cost @ \$75/hr
Responsive multi-user web portal (accounts, access control, audit, mobile)	160–220	\$12,000–\$16,500
Live OneSite catalog sync (310 reports, 10 pages) + property directory sync	60–90	\$4,500–\$6,750
Request workflow engine (parameters, Select-All preflight, queue)	80–120	\$6,000–\$9,000
Self-hosted GitHub Actions runner + live Edge session control layer	100–140	\$7,500–\$10,500
Download, filing, and idempotent reconciliation pipeline	70–100	\$5,250–\$7,500
Report library, PDF viewer, Excel preview, HTML companion generation	60–90	\$4,500–\$6,750
Manager's Checklist system (editable, auto-save, submit-for-review)	60–80	\$4,500–\$6,000
Email drafting workflow + contacts-directory matching	30–50	\$2,250–\$3,750
Windows Edge bridge (extension + native host + installer + tests)	80–120	\$6,000–\$9,000
Security architecture, credential isolation, staged validation, docs	60–80	\$4,500–\$6,000
Total	760–1,090 hrs	\$57,000–\$81,750

In calendar terms, that is roughly **4.5 to 6.5 months of one full-time senior developer** — or a \$60,000–\$80,000 fixed-bid engagement from any consulting shop, before ongoing maintenance. The expectation that this be "finished in less than a week" is not anchored to the actual scope of the system.

Part 2 — Every App and Portal Built In-House, and What Each Would Cost at \$75/Hour

The inventory below is drawn from my 3D portfolio (which catalogs the work portfolio), my GitHub repositories, and the live deployments. Hour estimates are conservative midpoints for a \$75/hr developer building the same scope, including design, development, testing, deployment, and documentation. Tools that are pure configuration of third-party platforms (the Notion contacts database, the Canva onboarding site) are listed at nominal value.

2.1 Flagship platforms (Tier 1)

#	Application	What it does	Est. hours	Cost @ \$75/hr
1	OneSite & Yardi Reporting Portal (OneSite Reporting Hub)	Full reporting automation platform detailed in Part 1 — catalog sync, request queue, self-hosted runner, filing pipeline, report library, manager checklists, Windows Edge bridge	760–1,090	\$57,000–\$81,750
2	PropertyMax.ai — Enhanced Manager Operations Hub (dashboardenhanced.manus.space (https://dashboardenhanced.manus.space/))	Intelligent action inbox with AI prioritization, Golden Score metrics, and a multi-property command center — an enhanced prototype of the company's own platform vision	200–280	\$15,000–\$21,000
3	Operations Hub (operationshub-aptcorp.manus.space (https://operationshub-aptcorp.manus.space/))	Central operations portal coordinating day-to-day property-management workflows; single entry point into the operations toolset	120–180	\$9,000–\$13,500
4	Vacancy Report WebApp (vacancyapp-42vudntx.manus.space (https://vacancyapp-42vudntx.manus.space/))	Real-time vacancy tracking across the portfolio: vacant/unleased/preleased/on-notice/eviction totals, daily building-by-building change tracking, 7-day trend views, one-click live refresh	100–150	\$7,500–\$11,250
5	Investor Summary Portal (investor-summary.manus.space (https://investor-	Investor-facing performance portal: search across all properties and investors by name, entity, or EIN;	100–150	\$7,500–\$11,250

#	Application	What it does	Est. hours	Cost @ \$75/hr
	summary.manus.space/))	Properties/Investors views; Missing vs. Complete K-1 filtering; full drill-down detail		
6	New Hire Onboarding Portal & Admin (new-hire-onboarding.manus.space (https://new-hire-onboarding.manus.space/admin))	End-to-end onboarding portal hosting and filing W-4, I-9, compliance documents, and handbooks into an organized server-side file ecosystem, with an admin console	150-220	\$11,250-\$16,500
7	PropertyMax.ai Training Hub (Phone Excellence Playbook & Certification)	Professional training hub with embedded instructional video, interactive knowledge-check quiz, downloadable playbook PDF, and automated completion-certificate generation	80-120	\$6,000-\$9,000

2.2 Operational and financial tools (Tier 2-3)

#	Application	What it does	Est. hours	Cost @ \$75/hr
8	Rent Comps API (rentcompsapi-kmyznwz7.manus.space (https://rentcompsapi-kmyznwz7.manus.space))	Live API serving rent comparables for 42 properties / 3,755 units from the master workbook, with Microsoft Graph/SharePoint integration for workbook sync	80-120	\$6,000-\$9,000
9	Rental Comps Portal (brandonrose2.github.io/claude-rent-comps (https://brandonrose2.github.io/claude-rent-comps/))	Rental comparables research portal for browsing comps across all properties for underwriting	40-60	\$3,000-\$4,500
10	Loss to Lease Reporting Portal (loss-to-lease.manus.space (https://loss-to-lease.manus.space/portfolio))	Loss-to-lease reporting with portfolio performance breakdowns and shareable report views	60-90	\$4,500-\$6,750
11	Cost Segregation Study Portal (brandonrose2.github.io/cost-segregation-project (https://brandonrose2.github.io/cost-segregation-project/))	Portal supporting cost-segregation studies across 10 apartment properties, including CPA-ready depreciation reconciliation schedules and workbooks	60-90	\$4,500-\$6,750
12	Monthly Inspections Tracker	Per-property monthly inspection	60-90	\$4,500-\$6,750

#	Application	What it does	Est. hours	Cost @ \$75/hr
	(monthlycheck-tukrskz5.manus.space (https://monthlycheck-tukrskz5.manus.space/))	checklists with status tracking across buildings and report/export controls		
13	Inspector Operations Suite (GitHub: inspector-operations-suite)	Portal hosting the inspection forms managers use for monthly unit inspections	50–80	\$3,750–\$6,000
14	Regional Manager Data Collection Portal (aptdash-2udxrvi.manus.space (https://aptdash-2udxrvi.manus.space/))	Collects multiple specified data points from all regional managers in one place, feeding portfolio-wide reporting	60–90	\$4,500–\$6,750
15	Property Directory / Org Chart (aptorgchart-8jqkzne.manus.space (https://aptorgchart-8jqkzne.manus.space/))	Central directory of all managed properties with an organizational chart, synced from the master Excel data and mirrored to Notion	40–60	\$3,000–\$4,500
16	Report Schedule Builder (marcreport-ypkstjhu.manus.space (https://marcreport-ypkstjhu.manus.space/))	Custom recurring-report schedule builder for regional teams with configurable cadence per report	40–60	\$3,000–\$4,500
17	Property Flyer Generator — two builds (Manus build (https://brandonrose2.github.io/propertymax-flyer-studio/) · Claude build (https://brandonrose2.github.io/propertymax-flyer-studio-claude/))	Standalone tool for creating, printing, and downloading property referral flyers from a simple form; built twice in parallel to compare AI development stacks	40–60	\$3,000–\$4,500
18	Referral Flyer WebApp (GitHub: referral-flyer-webapp)	Manager-facing portal that looks up property photos from a Drive folder by property selection	30–50	\$2,250–\$3,750
19	Property Marketing Asset Hub (GitHub: property-marketing-asset-hub)	Centralized internal marketing asset hub for multi-property management — property specs and downloadable assets	40–60	\$3,000–\$4,500
20	Batch File Editor / PDF Renamer (pdfrenamer-umxzpsmv.manus.space (https://pdfrenamer-umxzpsmv.manus.space/))	Batch file-editing and renaming utility that eliminates manual per-file processing	25–40	\$1,875–\$3,000
21	EDD Audit Preparation Portal (edd-audit-checklist)	Step-by-step EDD audit preparation checklist portal keeping audit prep	30–50	\$2,250–\$3,750

#	Application	What it does	Est. hours	Cost @ \$75/hr
	(https://brandonrose2.github.io/edd-audit-checklist/) + edd-audit-preparation-webapp)	organized and repeatable		
22	Portfolio Activity Overview Module (GitHub Pages (https://brandonrose2.github.io/portfolio-activity-overview/))	Executive module summarizing activity across the ApartmentCorp portfolio	40–60	\$3,000–\$4,500
23	CEO Command Center (GitHub: ceo-command-center / ceo-hub-site)	Executive reporting dashboard where the CEO can observe the health metrics of all properties	80–120	\$6,000–\$9,000
24	Tasks Backup Portal (tasks-dashboard (https://brandonrose2.github.io/tasks-dashboard/) + org repo AptCorp-PropertyMax/aptcorp-all-company-tasks-backup)	PIN-protected vault and dashboard backing up all team members' AI task history and case files	40–60	\$3,000–\$4,500
25	Property Manager Feedback & Training Request App	Internal app for managers to submit feedback and training requests	30–50	\$2,250–\$3,750
26	Company Contacts Directory (Notion)	Shared company contacts workspace used by the reporting portal for manager matching	8–12	\$600–\$900
27	New Hire Onboarding (Canva companion site + How-It-Works deck)	Companion onboarding walkthrough site and explanatory presentation	12–20	\$900–\$1,500

2.3 Personal/portfolio projects (listed for completeness, excluded from the company total)

These were built on my own initiative and are not counted toward the company figure: the **3D Portfolio Constellation** itself (a Three.js interactive 3D showcase of the work portfolio), the **Piano Composition Mastery portal**, the **AI Learning Portal**, the **résumé bio site**, and the **Shelene open-house listing site**.

2.4 The bottom line

Category	Hours	Value @ \$75/hr
Flagship platforms (7)	1,510–2,190	\$113,250–\$164,250
Operational & financial tools (20)	865–1,322	\$64,875–\$99,150

Category	Hours	Value @ \$75/hr
Total company software portfolio (27 applications)	≈ 2,375–3,512 hrs	≈ \$178,125–\$263,400

Conservative midpoint: roughly \$221,000 of custom software development — approximately 2,943 hours, or about 1.4 full-time developer-years — delivered in-house at \$75/hour equivalent, at no incremental cost to the company beyond my salary.

For calibration: the U.S. Bureau of Labor Statistics pegs median software developer pay near \$63/hr salaried, while freelance and agency rates for integration-heavy business software run \$75–\$200/hr [3] [4] [5]. The \$75/hr figure used here is therefore the *floor* of what this work would cost on the open market, not the ceiling.

Part 3 — The Ask

The OneSite & Yardi Reporting Portal is the largest single piece of that approximately \$221,000 software portfolio, and it is at a deliberate safety gate — not a stall. The immediate low-risk path is to repair the existing Mac runner, reconcile Request #60001 without rerunning it, and then begin the isolated Yardi catalog flow. A Windows runner remains a valid future option, but it is a separate security-reviewed build rather than a one-click substitution. This sequence ensures that nothing touches live provider data until every safeguard is verified, protecting the company's OneSite and Yardi accounts, resident PII, and financial reporting integrity.

I am happy to walk through the live portal, the runner repository, the reconciliation plan, or any of the 27 applications above in person. Every system listed is deployed, documented, and demonstrable today.

References

[1]: RealPage — OneSite property management platform (<https://www.realpage.com/>)

[2]: Yardi — property management software (<https://www.yardi.com/>)

[3]: U.S. Bureau of Labor Statistics — Software Developers, Occupational Outlook Handbook (<https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>)

[4]: Upwork — How much does it cost to hire a software developer (<https://www.upwork.com/hire/software-developers/cost/>)

[5]: Arc — Freelance software developer hourly rates (<https://arc.dev/freelance-developer-rates>)